

AI Agents That Decide, Grade, and Ship Their Own Code

A production AI system where large language models don't just answer questions — they **make operational decisions, grade their own outcomes in plain English, and ship their own code fixes** through a hard human-reviewed gate. Three cooperating agent layers on one audited Postgres backend, running unattended on a single server. Built end-to-end at TechInject.

Claude (Anthropic)

6 competing AI backends

Python · PostgreSQL · Streamlit

Cron · PM2 · nginx

~16,700 lines · 20-table schema

WHAT IT IS

An LLM-gated, self-improving, multi-agent automation platform — proven on intraday paper trading

AI INSIDE OPERATIONS

Claude calls every action, grades every outcome, and — as three agents — ships & verifies its own code

SCALE

~16,700 lines of Python · 20-table PostgreSQL · 10-page dashboard · 6 competing AI backends

RUNTIME

Cron-driven on one Linux VPS behind PM2 + nginx + Let's Encrypt — no queue, no Redis, no async layer

COST DISCIPLINE

POC runs at near-zero marginal LLM cost behind a one-env-var swap to paid production transport

STACK

Claude · Python 3.11 · PostgreSQL · Streamlit · cron · PM2 · nginx

The problem: automation without judgment is blind

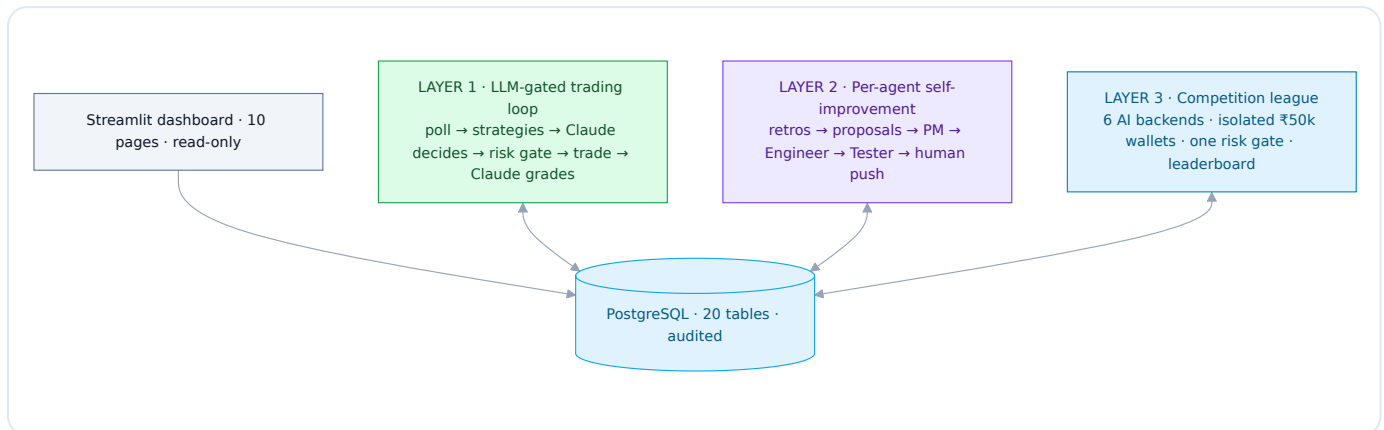
Most "automation" is a set of rules. A rule fires, the system acts, and nobody asks whether the action made sense until it has already cost something. That breaks down in three ways every operations team recognizes:

- **No judgment at the point of action.** A rule can't weigh context a human would — "this is a bad setup," "this lead is obviously junk," "this ticket is a duplicate." It just fires.
- **No accountable explanation afterward.** Once a decision is logged, there's no easy way to audit *why* it was made — especially for a non-technical operator reading the day's activity.
- **No path from lesson to fix.** Even when a mistake is spotted, turning it into a shipped improvement is manual, slow, and rarely happens.

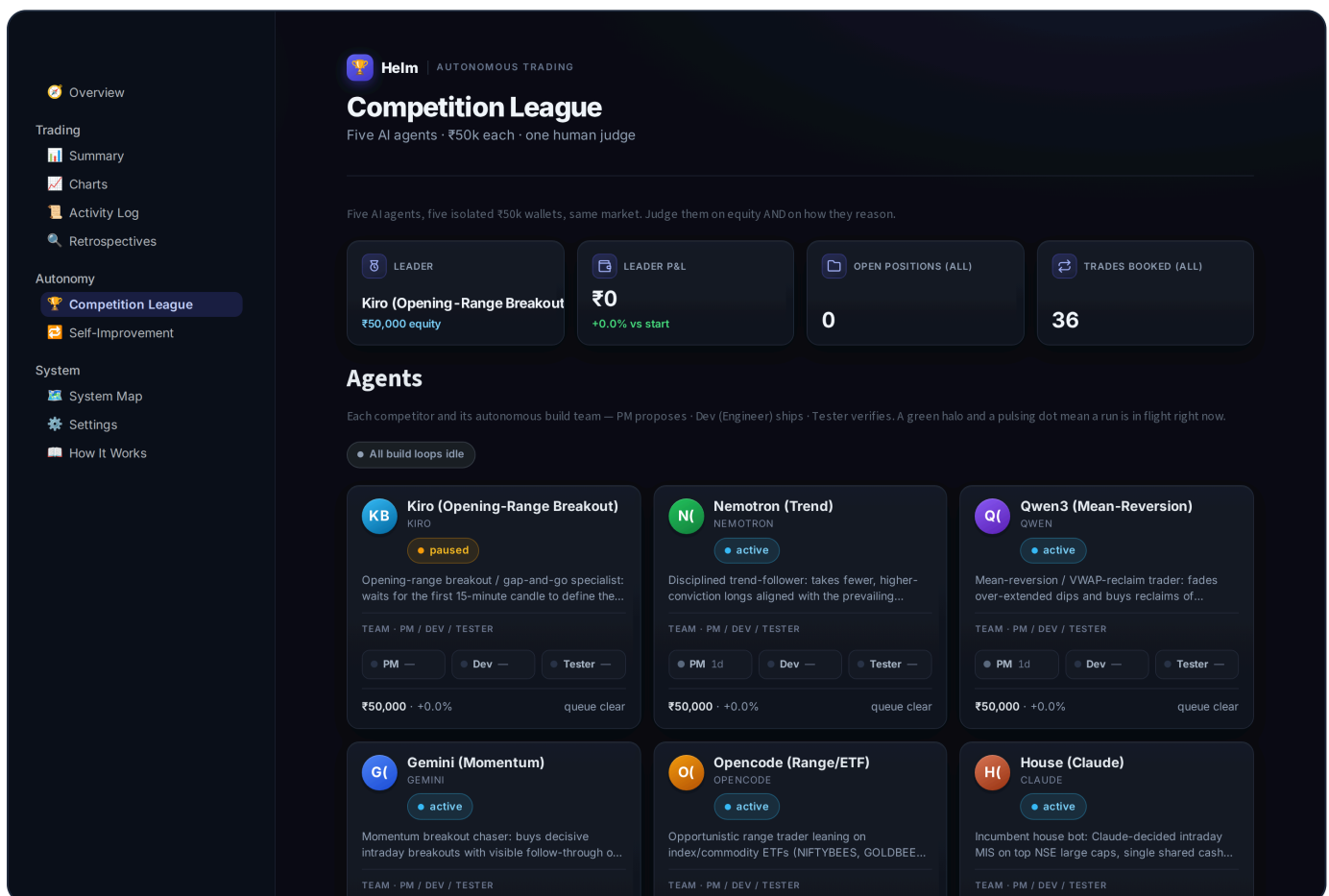
We kept the fast, deterministic, cheap rule layer **and** wrapped real intelligence around it: an AI judge in front of every action, an AI grader behind every outcome, and an AI engineering team that closes the loop from lesson to deployed change.

The solution: three AI agent layers on one backend

Helm is built in three layers that share a single PostgreSQL database, driven by cron jobs that no-op silently outside working hours — no fragile always-on workers.



Three AI agent layers sharing one audited Postgres backend, surfaced through one read-only dashboard.



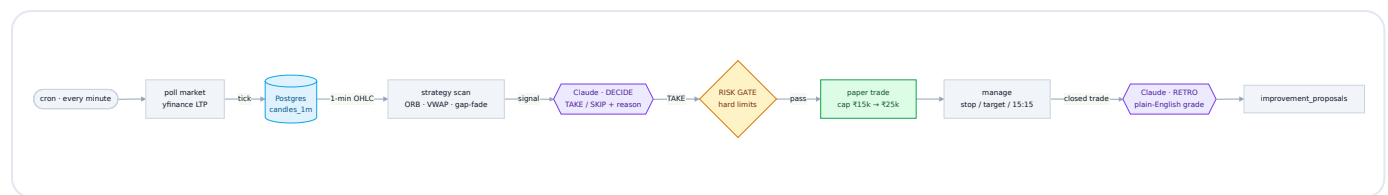
Live dashboard — the Competition League: six AI agents, each with a persona, status, equity, and its own PM/Dev/Tester team.

LAYER 1 **LIVE**

LLM as the decision gate

Mechanical strategies propose candidate actions. Before anything happens, **Claude reads the full context and rules TAKE or SKIP with one-line reasoning**. Every decision — including every SKIP and every risk-gate rejection — is written as its own database row, so nothing is unaccountable. On a TAKE, a *hard-coded* risk validator has the final word; the LLM can suggest,

never override the guardrails. After the fact, a **second Claude call grades the outcome in plain English** — a five-way verdict (good call / bad call / lucky / unlucky / mixed), concrete learnings, and quality scores a non-expert can read.



Layer 1 — signal → Claude decides → hard risk gate → trade → Claude grades. The model proposes; pure code disposes.

Overview

Trading

Summary

Charts

Activity Log

Retrospectives

Autonomy

Competition League

Self-Improvement

System

System Map

Settings

How It Works

From 2026/05/11 To 2026/05/25 Kind All Verdict Choose options

> Run the retro cron now

TOTAL REVIEWS 116

GOOD CALL 72

UNLUCKY 32

LUCKY 0

BAD CALL 8

MIXED 4

Reviews

← Prev Page 1 / 8 · 1-15 of 116 reviews Next →

RELIANCE · SKIP · passed · Bad call · 22 May 03:54:39 PM IST

Bad call

Summary

The bot spotted a genuinely good buying opportunity in Reliance but could not act on it because the paper-trading wallet only had ₹790 left — not enough to buy even a single share priced at ₹1358. The price did exactly what the setup predicted, reaching the profit goal of ₹1366 by 1:04 PM and earning what would have been ₹8 per share.

Why we acted

The bot's analysis was actually positive: the price had been climbing steadily for five straight minutes into the buy level, all risk checks were green, and there was plenty of time left in the trading day. The skip was not a judgment call — it was a hard block imposed at the very last step because the wallet balance of ₹790 fell short of the single-share cost of ₹1358 by ₹568.

What happened

After the signal at 11:43 AM, Reliance continued to rise smoothly, crossing ₹1359 within minutes and grinding upward over the next hour. The profit target of ₹1366 was hit at 1:04 PM — a clean, unhurried winner. Notably, the price then reversed sharply in the afternoon, falling to ₹1352 by 3:14 PM, which would have been below the planned stop-loss — so the original exit plan was well-timed.

Verdict reasoning

The bot's own analysis judged this setup as worth taking, and the price proved it right by hitting the profit target without ever threatening the stop. A skip that lets a clean winner pass is a BAD_CALL. The root cause is a system failure — an empty wallet — not a flaw in the trade idea or the AI's analysis.

Learnings

- The paper-trading wallet ran so low that it could not afford a single share of Reliance. When a wallet is too small to buy any meaningful quantity of a stock on the watchlist, that stock should be excluded from scanning until the wallet is replenished — not discovered at the last step after full analysis.
- The price hit its profit goal at 1:04 PM and then reversed all the way back toward the stop-loss level by mid-afternoon. The original exit plan — take profit at ₹1366, stop at ₹1353 — was

Sample AI output — a closed-call retrospective: plain-English summary, why we acted, what happened, verdict reasoning, and learnings, auto-written by Claude. Here a SKIP is graded a "bad call."

LAYER 2 BUILT

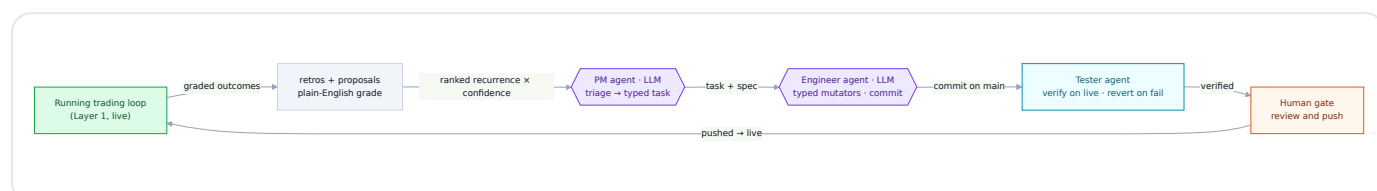
Agents that improve the system itself

Graded mistakes don't sit in a report nobody reads. They become a ranked queue of improvement ideas that **three cooperating AI agents** turn into shipped change — and the loop runs **per competitor**, so every agent in the league has its own PM, Engineer, and Tester working only on its backlog:

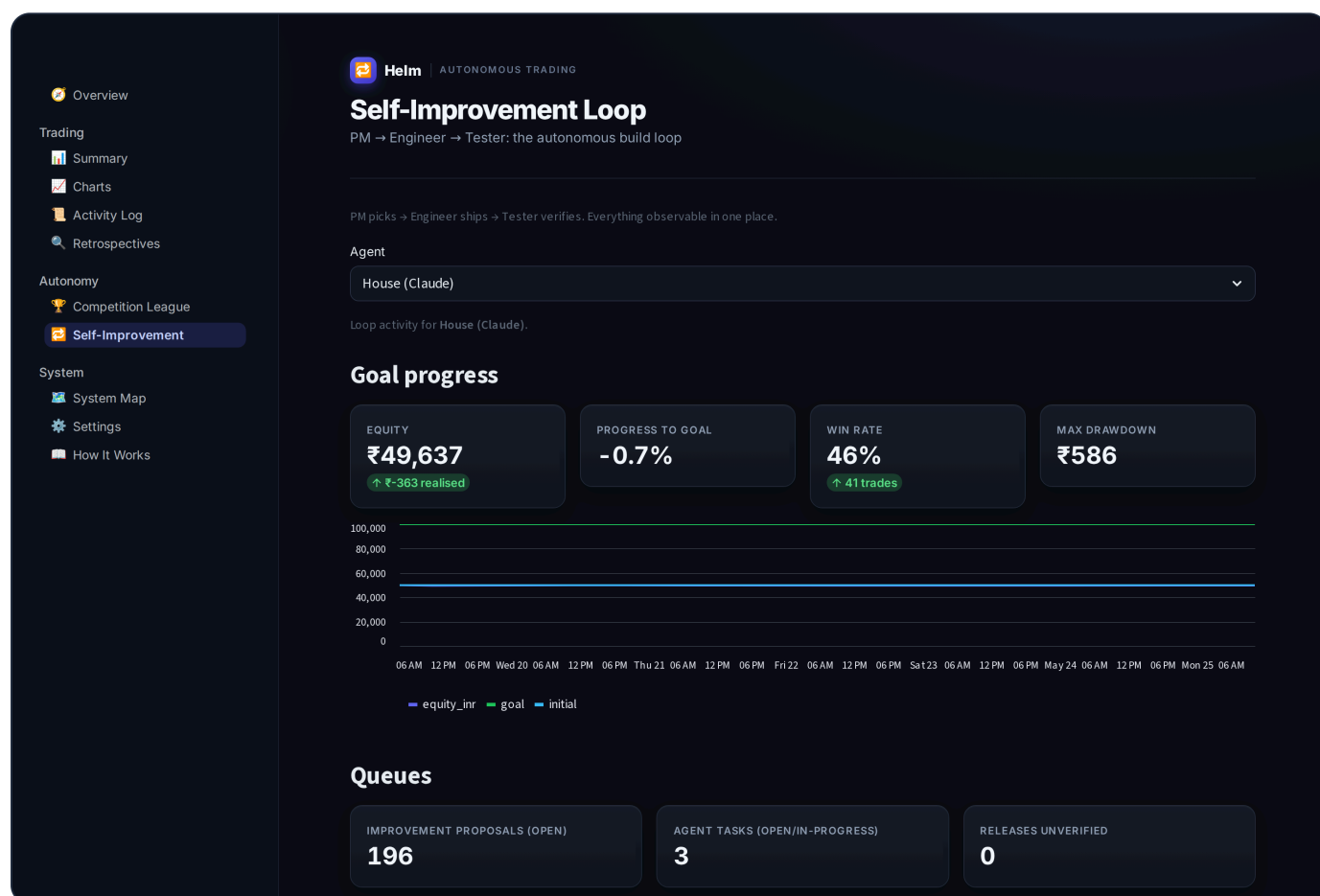
- A **Product Manager agent** triages the queue against live metrics into well-scoped typed tasks — recurrence beats novelty, cheapest reversible fix first — and runs a **semantic consolidation pass** that collapses hundreds of near-duplicate proposals into a deduped backlog.

- An **Engineer agent** executes one task at a time. For the house agent: a *closed set of typed code mutators* (never free-form coding), lint and tests as a hard gate, then a commit to main. For a freestyle competitor: tuning that agent's own persona and config. Anything outside the safe surface is flagged for a human.
- A **Tester agent** verifies every change on the live deployment through a six-stage check and **reverts automatically** if anything regresses, keeping main always green.

The human is the **only push gate** — the Engineer commits locally, a person reviews and pushes. Two reverts in a row auto-pause the whole loop. This is the differentiator most "AI agent" demos never reach: agents that operate safely against a real, running system.



Layer 2 — the always-green PM → Engineer → Tester loop, run independently for every agent, with a human as the only push gate.



Self-Improvement Loop page — a per-agent selector, goal-progress chart (equity vs goal), and the open proposal / task / release queues.

LAYER 3 **BUILT**

A multi-agent competition league

To compare AI models on the *same* job, Helm runs six AI backends — **Claude (the house incumbent)**, **Gemini**, **Qwen3**, **Nemotron**, **opencode**, and **Kiro** — each trading an isolated ₹50,000 paper wallet, each racing to **double it**. Every freestyle agent gets a distinct trading

persona — Momentum, Mean-Reversion, Trend, Range/ETF, Opening-Range Breakout — so the league tests judgment styles, not just model names. They all flow through the *same* shared, quota-controlled, risk-gated execution path, so the only variable is the model's own judgment. A leaderboard ranks them on a common basis, a "how it thinks" view shows each agent's raw prompt and reasoning, and each competitor renders as a card with its avatar, live status, equity, and its own PM/Dev/Tester team. This is multi-agent orchestration with real isolation, per-backend rate-limit handling, and auto-resume — not six chatbots in a trench coat.



Layer 3 — six AI backends, one shared risk-gated path, isolated wallets, a common leaderboard.

Overview

Trading

Summary

Charts

Activity Log

Retrospectives

Autonomy

Competition League

Self-Improvement

System

System Map

Settings

How It Works

Helm

AUTONOMOUS TRADING

Retrospectives

What the bot learned from every call

After every closed trade and every skipped idea, the bot writes a plain-English review: why we acted, what happened, whether it was a good call, and what to remember for next time.

Agent

House (Claude)

Showing retrospectives for House (Claude).

From

To

Kind

Verdict

2026/05/11

2026/05/25

All

Choose options

> Run the retro cron now

TOTAL REVIEWS

GOOD CALL

UNLUCKY

LUCKY

BAD CALL

MIXED

116

72

32

0

8

4

Reviews

< Prev

Page 1 / 8 · 1-15 of 116 reviews

Next >

> RELIANCE · SKIP · passed · Bad call · 22 May 03:54:39 PM IST

> KOTAKBANK · SKIP · passed · Unlucky · 22 May 03:53:07 PM IST

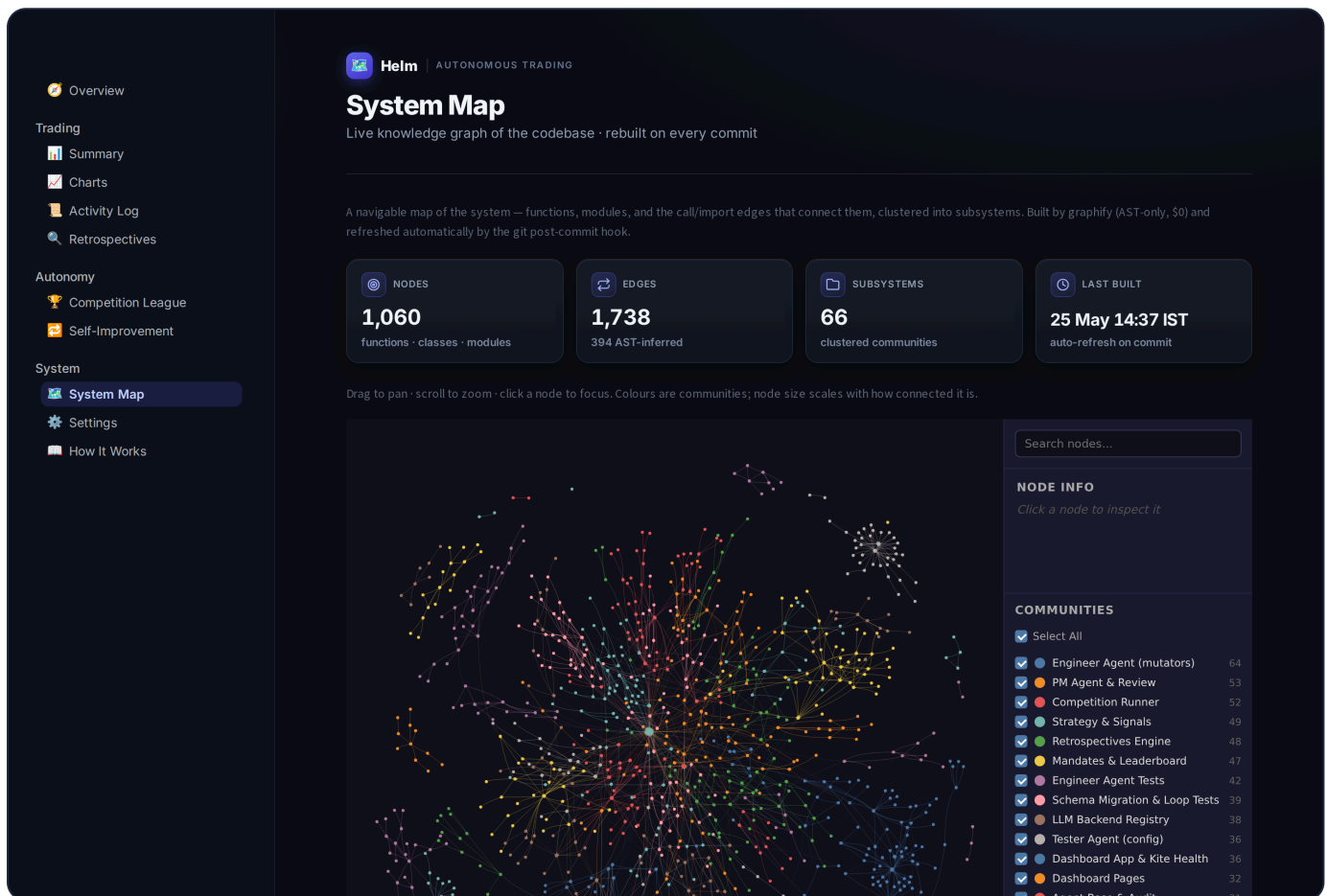
Retrospectives page — the five-way verdict distribution across every reviewed call, filterable per agent. SKIPs are graded against a counterfactual replay, so "bad call" and "unlucky" are real judgments, not the P&L sign.

The engineering decisions that make it production-grade

Anyone can wire an LLM to an API. The reason this runs unattended without babysitting is the boring infrastructure around it:

- Idempotency and dedup everywhere.** A consumed flag flips inside the same transaction as the decision row, so the AI can never act twice on the same item; retrospectives are unique-indexed so a re-run can't double-grade.
- Race-proofing.** Postgres advisory locks and FOR UPDATE SKIP LOCKED task claiming mean overlapping cron firings never collide or burn duplicate LLM calls.

- **A hard validator the AI can't bypass.** Risk limits are pure code, always the final word.
- **Tolerant parsing.** When the model wraps JSON in markdown fences, a forgiving parser handles it instead of crashing the pipeline.
- **Plain-English by mandate.** The grader's prompt carries a banned-jargon list that forces every explanation into language a non-expert can use.
- **Cost control as an interface.** All LLM traffic sits behind one function with swappable backends — the POC runs at near-zero marginal cost and flips to a paid production transport with a single environment variable, no code changes.
- **The system maps itself.** A live **System Map** turns the whole codebase into a navigable knowledge graph — ~1,060 functions/classes/modules and ~1,700 import edges in ~66 clustered subsystems — rebuilt automatically on every commit, so the architecture docs can't drift from the code.



System Map — the codebase rendered as a live, clustered knowledge graph, rebuilt on every commit.

The results

- **Runs unattended** through the trading session every weekday and silently no-ops outside the window — no babysitting, no log spam.
- **Every decision is auditable.** Every signal yields exactly one explained decision; every closed action yields exactly one plain-English grade.
- **The system improves itself** within a tightly bounded, human-gated, always-green workflow — changes ship and self-verify, or revert automatically.
- **Six AI models compete head-to-head** (five live, one queued) on identical infrastructure with fully isolated capital, per-model quota safety, and a self-improvement loop each.

- **Deployed on a single server** behind PM2, nginx, and Let's Encrypt — a deliberately small, debuggable footprint with no queue, no Redis, no async sprawl.

Where this applies to your business

The trading domain is just the proving ground. The reusable asset is the **architecture**:

Deterministic producers propose → an AI judges the action → a hard validator guards → a second AI grades the outcome → agents improve the loop.

That scaffold maps directly onto:

- **AI agents over your CRM / ops data** — read records, qualify leads, draft outreach, update fields, with every action logged and explained.
- **Document & data automation** — invoices, contracts, and reports parsed by an LLM and routed, with a hard validation gate before anything is committed.
- **Support-ticket triage & content moderation** — high-volume decisions that need pre-action judgment *and* an after-the-fact, human-readable rationale.
- **Replacing fragile no-code flows** — swap brittle Zapier/Make/n8n chains for a proper service with idempotency, retries, audit logs, and cost controls a team can actually trust.

Tech stack

AI / AUTOMATION	Claude (Anthropic API + CLI); multi-backend LLM orchestration — Gemini, Qwen3, Nemotron, opencode, Kiro
BACKEND	Python 3.11, PostgreSQL, cron
INTERFACE	Streamlit dashboard (10 pages, read-only over Postgres)
OPS	PM2, nginx, Let's Encrypt, single Linux VPS

TechInject builds production AI agent systems on real backends — not no-code prototypes. Live operator dashboard and a code walkthrough available on request. Research / paper-trading system; nothing here is investment advice.